

CS801: Advanced Distributed Systems & Consensus Protocols

Department of Computer Science & Engineering - Technical Lecture Notes

1. Foundations of Distributed Systems

A distributed system is defined as a collection of autonomous computing entities (nodes) that communicate over a shared network to coordinate actions and share resources.

Unlike centralized architectures, distributed environments inherently face partial failures, asynchronous network latencies, and lack of a globally synchronized physical clock. As defined by Lamport, a distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

1.1 The CAP Theorem and Consistency Models

Formulated by Eric Brewer and formally proved by Seth Gilbert and Nancy Lynch, the CAP theorem states that any distributed data store can simultaneously provide at most two of the following three guarantees: Consistency (every read receives the most recent write or an error), Availability (every non-failing node returns a non-error response), and Partition Tolerance (the system continues to operate despite arbitrary packet drops or network partitioning). Under real-world physical networks where partitions are inevitable, system designers must choose between CP (consistent under partition) and AP (available under partition).

1.2 Linearizability vs Sequential Consistency

Linearizability refers to a strong recency guarantee where operations appear to take effect instantaneously at a specific point in real time between their invocation and response. Sequential consistency, by contrast, relaxes real-time ordering and requires only that all processes agree on a single global sequence of operations that preserves each process's local program order.

2. The Consensus Problem and FLP Impossibility

Consensus is defined as the core coordination primitive whereby distributed nodes agree upon a single data value or sequence of state machine transitions. A consensus protocol must satisfy three correctness properties: Agreement (all non-faulty nodes decide the same value), Validity (the decided value must have been proposed by some node), and Termination (all non-faulty nodes eventually decide a value).

2.1 The FLP Impossibility Result

The landmark theorem proved by Fischer, Lynch, and Paterson (1985) establishes that in an asynchronous network model, no deterministic consensus protocol can guarantee termination in the presence of even a single unannounced crash failure. Because message delivery delays are unbounded, a node cannot distinguish between a failed node and an exceptionally slow network connection. Consequently, real-world consensus protocols such as Paxos, Raft, and PBFT make partial synchrony assumptions or use randomized timeouts.

2.2 Foundational Prerequisites

To understand consensus mechanisms, students require prior mastery of several foundational concepts: Discrete Mathematics (graph structures and partial orders), Network Sockets (TCP packet transmission, sliding windows, and retransmission timeouts), and State Machine Replication (formal deterministic automata transitioning across identical logs).

3. Raft Consensus: Architecture and Decomposition

Designed by Ongaro and Ousterhout at Stanford University, Raft decomposes consensus into three independent sub-problems: Leader Election, Log Replication, and Safety invariants. Nodes in Raft exist in one of three states: Follower, Candidate, or Leader. Time is divided into discrete Terms numbered with monotonic integers.

3.1 Leader Election and Randomized Heartbeats

When followers experience heartbeat timeout, they increment their term and transition to candidate status, broadcasting RequestVote RPCs. To prevent split-vote scenarios where multiple candidates split the quorum, Raft employs randomized election timeouts (e.g., 150ms to 300ms). A candidate wins the election once it receives votes from a strict majority quorum of $(N/2 + 1)$ nodes.

3.2 Log Matching and State Machine Invariant

Log replication occurs via AppendEntries RPCs. If two entries in different logs have the same index and term, they store the same command, and their logs are identical in all preceding entries. The Leader Completeness property guarantees that if a log entry is committed in a given term, that entry will be present in the logs of the leader for all higher terms.

4. Byzantine Fault Tolerance (BFT) and Practical Applications

A Byzantine fault refers to an arbitrary or adversarial failure where nodes can send conflicting messages, forge identities, or intentionally collude against system safety. The classic Byzantine Generals Problem established that in synchronous networks, reaching consensus requires strictly more than $3m + 1$ total nodes to tolerate m Byzantine traitors.

4.1 Practical Byzantine Fault Tolerance (PBFT)

Castro and Liskov (1999) introduced PBFT, the first practical algorithm achieving state machine replication under Byzantine conditions in asynchronous networks with polynomial time complexity $O(R^2)$. PBFT processes requests through a three-phase commit pipeline: Pre-Prepare, Prepare, and Commit, relying on cryptographic digest signatures to guarantee non-repudiation.

4.2 Comparative Summary of Consensus Protocols

In production systems, Paxos and Raft are standard for crash-fault-tolerant (CFT) enterprise storage like etcd, Consul, Apache Kafka, and CockroachDB, operating with $2f + 1$ nodes. For untrusted, adversarial environments, BFT protocols like PBFT, Tendermint, and HotStuff ensure safety with $3f + 1$ nodes.